

Problem Statement: Life RPG

1. The Core Problem

Traditional productivity tools, habit trackers, and to-do lists are fundamentally flawed for many users: they feel like chores. They suffer from a "delayed gratification" problem, where the real-world results of reading a book, going to the gym, or studying take months to materialize. In contrast, video games provide immediate dopamine through instant feedback loops, clear progression systems, and tangible rewards.

Your objective is to bridge this gap by building a **Life RPG web application**. You must engineer a platform that translates mundane real-world tasks into an engaging virtual progression system.

Unlike a simple frontend prototype, this project requires a robust full-stack architecture. It needs a secure backend to prevent users from easily "cheating" their stats, a relational or document database to maintain complex historical logs of completed tasks and inventory, and user authentication to allow seamless cross-device synchronization.

2. Product Feel & Creative Direction

This application should not look or feel like a standard enterprise SaaS dashboard or a generic Bootstrap CRUD app. It needs a soul. You have complete creative freedom over the thematic execution—whether you build a cozy lo-fi study room, a 16-bit retro dungeon crawler, a sleek cyberpunk interface, or a minimalist modern dashboard.

However, regardless of the theme, the application must feel:

- **Alive and Tactile:** The UI should react instantly to the user. Earning Experience Points (XP), leveling up, or buying an item should feel celebratory. Use CSS micro-interactions, spring animations, or subtle particle effects to make mundane checkmarks deeply satisfying.
- **Thematically Cohesive:** If you choose a fantasy theme, your typography, color palette, and language (e.g., "Quests" instead of "Tasks", "Gold" instead of "Points") should match. Establish a strong visual hierarchy so the screen doesn't become cluttered with stats.
- **Seamlessly Integrated:** Even though data is stored on a remote server, the user should never feel bogged down by network latency. Utilize loading skeletons, optimistic UI updates, and smooth transitions so the experience feels as fast as a native client-side app.

3. Allowed Technology Pool

You may freely mix and match from the following technologies. **You are encouraged to use whatever full-stack architecture you are most comfortable with.**

- **Frontend / UI:** React, Vue, Svelte, Angular, SolidJS, HTML/CSS/JS. (Tailwind, SCSS, Radix, Shadcn, Framer Motion, GSAP, etc.)
- **Backend / API:** Node.js (Express/Nest), Python (Django/FastAPI), Go, Ruby on Rails, Java (Spring), Next.js API Routes, or SvelteKit endpoints.
- **Database (Relational or NoSQL):** PostgreSQL, MongoDB, MySQL, SQLite, Redis.
- **Backend-as-a-Service (BaaS) / Auth:** Firebase, Supabase, Appwrite, Clerk, NextAuth, or custom JWT/Session implementations.

4. Required Deliverables

1. **Public GitHub Repository:** Must contain all source code (frontend and backend), a clean commit history, and a detailed `README.md` containing setup instructions and environment variable templates (`.env.example`).
2. **Live Deployed URL:** A fully functioning, publicly accessible live link (e.g., hosted via Vercel, Netlify, Render, Railway, Heroku, or AWS).
3. **Illustration Video:** A screen recording (**under 100MB**, strictly 90–180 seconds) demonstrating user signup/login, adding/completing a task, the leveling up process, and a page refresh to prove database persistence. Must be hosted in the repo or via an accessible public link.

5. Core Features Checklist

To give you creative freedom, we are not dictating the exact math or specific items. However, your application **must** successfully implement the following core systems:

- **User Authentication & Security:** Secure signup, login, and session management. A user should only see and modify their own tasks and character data.
- **Database Schema & CRUD:** A thoughtfully designed database structure to handle Users, Tasks, and character attributes. Users must be able to Create, Read, Update, and Delete tasks smoothly.
- **The RPG Progression Engine:** Implement a non-linear leveling system (where each subsequent level requires more XP than the last).
- **Gamified Elements:**
 - **Streaks:** A system that tracks consecutive days of activity.
 - **Attributes:** Categorize tasks so they level up specific character stats (e.g., Coding increases "Intellect", Gym increases "Strength").
 - **Rewards/Economy:** A system where users earn currency or points to "buy" virtual items, themes, or profile badges.

- **Responsive & Accessible UI:** The app must be fully responsive (mobile to desktop screens), entirely navigable via keyboard (Tab, Enter, Space), and structurally sound for screen readers.

6. Disqualification Rules (Zero-Tolerance)

Submissions will receive an immediate zero if they violate any of the following:

Rule	Description
Broken Links	The GitHub repo is private, or the live deployment link is broken/inaccessible at the time of judging.
Fake Data Persistence	The application relies solely on <code>localStorage</code> for primary data. If user data does not persist across sessions, it is considered fake.
Build/Deployment Failure	The live app crashes on load, or the backend API fails to connect to the database in the production environment.
Console/Runtime Crashes	Unhandled runtime exceptions or blank-screen crashes occur during basic application usage.
Invalid Repository	The repo contains fewer than 3 chronological commits, lacks the backend code, or shows evidence of being a fork without original work.
Missing/Restricted Video	The walkthrough video is omitted, requires login authorization to view, or exceeds the 100MB limit.

7. Judging Criteria

Your submission will be evaluated based on the following pillars:

- **Design & UX (Crucial Warning):** The visual appeal and user experience are major factors. **Submissions with lazy, overly generic designs, default unstyled components, or poor visual hierarchy will fetch significantly lower marks.** We are looking for polish, creativity, and a cohesive theme.
- **Performance & SEO:** Fast load times, optimized assets, responsive performance across devices, semantic HTML, accessible structure, and SEO-friendly metadata/content to ensure a smooth user experience and strong search visibility.
- **Creativity & Gamification:** How creatively have you implemented the RPG elements? Does the progression system feel rewarding and well-thought-out, or does it feel like an afterthought?
- **Robustness & Edge Cases:** Does the app handle errors gracefully? What happens if a user submits an empty task, or their internet connection drops?
- **Accessibility & Responsiveness:** Does the application work flawlessly on mobile devices? Can it be navigated via keyboard?